

02 - HTTP Verbs

📅 09-09-2026 ⌚ Ongeveer 25 minuten 🗨️ Ongeveer 3721 woorden

GET, POST, PUT & DELETE

Nu je 'digitale ober' (de Controller) klaarstaat en je weet hoe je een bestelling opneemt, gaan we kijken naar de verschillende soorten opdrachten die een client kan geven. In de echte wereld praten we over `CRUD` : Create, Read, Update, Delete. In de webwereld vertalen we dit naar `HTTP Verbs` .

In dit hoofdstuk gaan we dieper in op de techniek achter deze verzoeken. Omdat we nog geen database hebben, gebruiken we een In-Memory lijst. Dit is een tijdelijke lijst in het geheugen van de server.

Terugblik & Focus

In het vorige hoofdstuk hebben we de fundamenten gelegd. We hebben kennisgemaakt met de 'digitale ober' (de Controller), de basis van routing ontdekt en onze interactieve menukaart (/scalar) opgezet in .NET 10.

Nu is het tijd om daadwerkelijk bestellingen op te nemen. In dit hoofdstuk brengen we de API tot leven door de volledige `CRUD`-cyclus te implementeren via `HTTP Verbs` (GET, POST, PUT, DELETE). We introduceren Models als datablauwdruk en leren professioneel communiceren met de client via universele `HTTP`-statuscodes en `ActionResult<T>`.

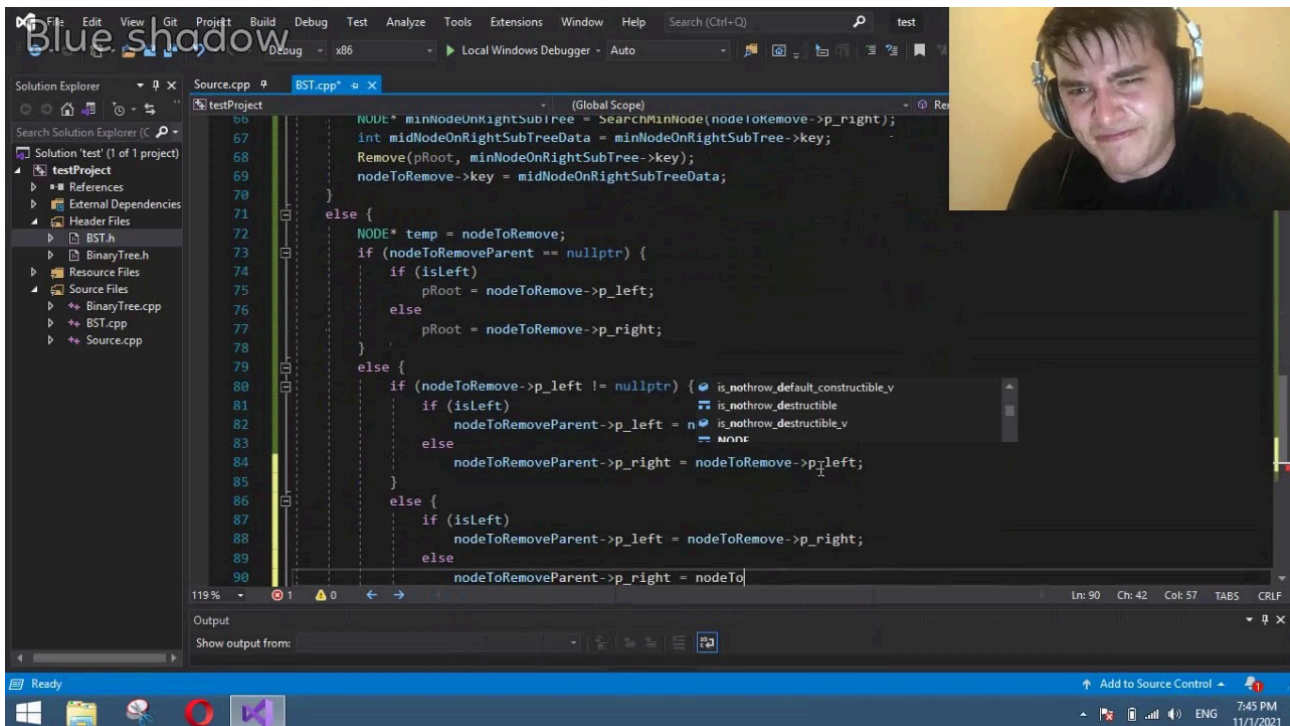
Models: De Blauwdruk van je data

Modellen zijn de ruggengraat van je API. Het zijn *code-representaties van objecten en concepten* uit de echte wereld: bestellingen, klanten, producten, of zelfs vijanden in een game.

In de vorige modules focusten we ons vaak op de primitieve datatypes zoals `int`, `string` en `bool` . Nuttig, maar te klein om een complex concept te beschrijven. Stel je voor dat je de laptop beschrijft waar je nu achter zit. Kun je dat met één `int`? Nee. Probeer je het met één lange `string`? Dan krijg je een onleesbare soep van tekst:

```
Dell; XPS13; 16GB; Intel I7; 1500.00; RTX4070".
```

Als je later enkel de prijs wilt verhogen voor een inflatiecorrectie, moet je die hele tekst-string uit elkaar gaan pluizen. Dat is vragen om fouten.



Modellen - Van losse data naar een Object

In C# gebruiken we een `Model` om kenmerken logisch te groeperen. Een Model is in feite een eenvoudige `class` die dienstdoet als een digitale bouwtekening. Vaak beschrijft een Model een real-world object of concept zoals een Persoon, een Verkoop, een Auto, een afspraak,... We gebruiken hiervoor `Properties` (eigenschappen).

Maak in je project een map aan genaamd `Models` en voeg daar de klasse `Laptop.cs` aan toe:

```

1  namespace WebAPIDemo.Models;                                     csharp
2
3  public class Laptop
4  {
5      // De 'id' (Primary Key) is essentieel om dit specifieke object later
6      terug te vinden
7      public int Id { get; set; }
8
9      public string Merk { get; set; } = "";
10
11     public string Processor { get; set; } = "";

```

```
12     public int RamInGB { get; set; }
13
14     public double Prijs { get; set; }
15
16     public string GPU { get; set; } = "";
17 }
```

Vaak is een Model niet meer dan dit. Een **eenvoudige klasse met Properties die een object beschrijven**. Modellen kunnen methodes hebben maar vaak zijn dit maar eenvoudige methodes die enkel betrekking hebben op *data* zoals iemand's leeftijd berekenen op basis van een geboortedatum of valuta omzetten van dollar naar euro. Business beslissingen / logica horen hier meestal niet thuis.

Tips

Gebruik de snippet `prop+tab+tab` om snel een Property te genereren.

Waarom is een Model de standaard in API-land?

- **Voorspelbaarheid:** De API weet precies wat hij kan verwachten. Als een methode om een Laptop vraagt, krijgt hij niet zomaar een getal, maar een object met een merk, prijs en processor.
- **Flexibiliteit:** Je kan later aan je model extra properties toevoegen of weer weghalen. Methodes die een laptop model verwachten hoeven vaak niet aangepast te worden. Alle data zit in het model.
- **Validatie:** Je kunt regels koppelen aan je model. Bijvoorbeeld: de prijs van een laptop mag nooit lager zijn dan €0.
- **Eenvoudige vertaling naar JSON:** Dit is cruciaal. Een API stuurt geen C#-code over het internet, maar `JSON (JavaScript Object Notation)`. .NET is slim genoeg om jouw Model automatisch om te zetten naar dit tekstformaat.

Wanneer jouw C# Model wordt verstuurd, kan het resultaat in JSON formaat er als volgt uitzien. Vrijwel elke programmeertaal (*JavaScript, Python, Swift, Swift, Java*) kan dit formaat moeiteloos lezen. Hiermee is je API dus direct universeel inzetbaar.

```
1  {
2      "id": 1,
3      "merk": "Dell",
4      "processor": "Intel i7",
5      "ramInGB": 16,
6      "prijs": 1499.99,
7      "gpu": "RTX 4070"
8  }
```

json

Info

In deze cursus zul je merken dat we voor elk concept dat we beheren (gebruikers, producten, bestellingen) een eigen Model maken in de map Models. Het is de ruggengraat van je API: zonder een goed model weet je applicatie niet over welke data hij praat.

Product Controller aanmaken

Nu we een Model hebben kunnen we een Controller aanmaken dat 1 of meerdere van deze modellen aan de client kan tonen.

Maak een nieuwe Controller aan in de map Controllers en noem deze **ProductController.cs**.

Omdat we nog geen echte database hebben, gebruiken we een lokale in-memory `List<Laptop>` .

```
1  using Microsoft.AspNetCore.Mvc;
2  using WebAPIDemo.Models;
3
4  namespace WebAPIDemo.Controllers;
5
6  [ApiController]
7  [Route("api/[controller]")] // Route wordt: api/product
8  public class ProductController : ControllerBase
9  {
10     // Onze tijdelijke "database" in het geheugen
11     private List<Laptop> laptops = new List<Laptop>
12     {
13         new Laptop { Id = 1, Merk = "Dell", Processor = "Intel i7",
14         RamInGB = 16, Prijs = 1200.00, GPU = "Iris Xe" },
15     }
```

csharp

```
15         new Laptop { Id = 2, Merk = "Apple", Processor = "M3 Pro",  
16         RamInGB = 18, Prijs = 2500.00, GPU = "Apple GPU" },  
17         new Laptop { Id = 3, Merk = "Lenovo", Processor = "AMD Ryzen 7",  
         RamInGB = 32, Prijs = 1400.00, GPU = "RTX 4060" }  
        };  
    }
```

> 🤖 A.I.: Laat een AI-assistent naar keuze een tiental laptopobjecten voor je genereren.

Tips

Heb je **GitHub Copilot** geïnstalleerd in VS Code? Gebruik dan de sneltoets `Ctrl + I` om de chat-assistent direct in je code-editor te openen.

HttpGet - Data ophalen

Deze data willen we nu beschikbaar stellen aan onze gebruikers. Net zoals in het vorige hoofdstuk zullen we een `HTTPGET` methode aanmaken om deze volledige lijst aan de gebruiker te geven.

Maak in de Controller een nieuwe methode aan genaamd **GetLaptops** en decoreer deze met de `[HttpGet]` attribuut zodat .Net weet dat deze methode bereikbaar is via HTTP en test deze uit met `Scalar`.

```
1 // 1. GET: Alle laptops ophalen  
2 // Route: GET api/laptops  
3 [HttpGet]  
4 public List<Laptop> GetLaptops()  
5 {  
6     return laptops;  
7 }
```

csharp

Tips

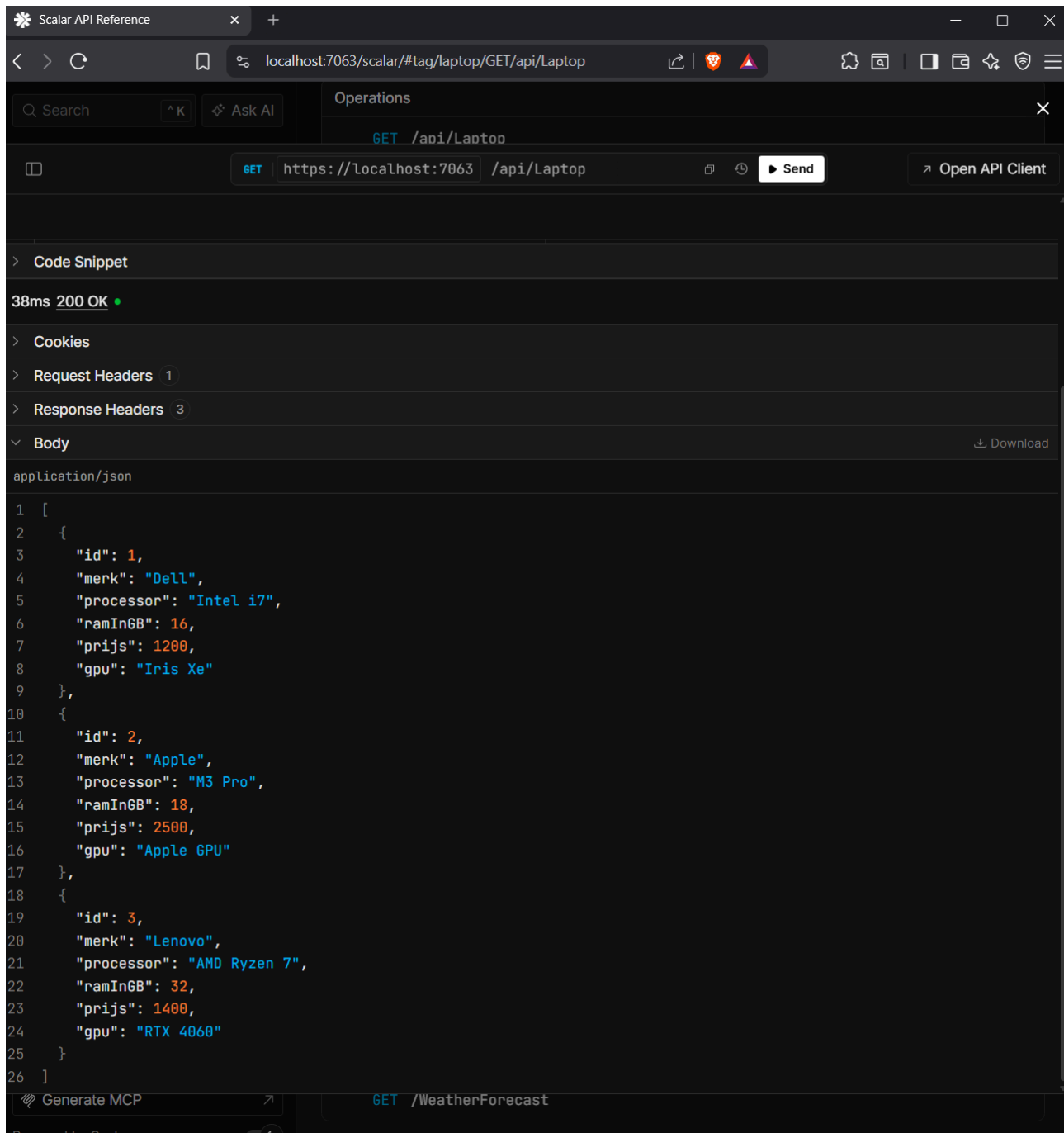
Wil je graag je endpoint benoemen, kun je een naam toevoegen aan de decorator.

```
1 [HttpGet("GetAll")]
2 public List<Laptop> GetLaptops()
3 {
4     return laptops;
5 }
```

CS

Endpoint: localhost:poortnr/api/Laptop/GetAll

Als je de applicatie start, kun je met Scalar testen of je gegevens worden opgehaald.



Daarnaast willen we ook een methode voorzien om een enkel laptop object op te halen. We gebruiken hiervoor de kennis uit de voorgaande modules om een enkel object uit een lijst op te halen aan de hand van de ID parameter.

Om de ID uit de route te halen voegen we "{id}" toe aan de `HttpGet` decorator. Dit zorgt ervoor dat we 2 verschillende routes (bv. `api/laptop` & `api/laptop/3`) voorzien die elk een `HttpGet` commando toegankelijk stellen.

```
1 // 2. GET: Eén specifieke laptop op basis van Id csharp
2 // Route: GET api/laptops/1
3 [HttpGet("{id}")]
4 public Laptop GetLaptopById(int id)
5 {
6     // Zoek de laptop in de lijst die de gevraagde Id heeft
7     Laptop laptop = laptops.FirstOrDefault(x => x.Id == id);
8
9     return laptop;
10 }
```

Info

🧠 **Opfrisser:** Wat was `FirstOrDefault()` ook alweer? Dit is een veelgebruikte methode binnen LINQ om door collecties te zoeken. Het geeft het **eerste** (`First`) element terug dat voldoet aan een bepaalde voorwaarde (in ons geval: waar de `Id` gelijk is aan de parameter). Wordt er niets gevonden? Dan geeft de methode `NULL` (`OrDefault`) terug.

Je kunt naast `FirstOrDefault()` ook `First()` gebruiken. Dit werkt qua zoektocht exact hetzelfde, maar met één belangrijk verschil: als er geen item wordt gevonden, crasht je applicatie met een **Exception** in plaats van `null` terug te geven.

En hoe zit het met `Single()` en `SingleOrDefault()` ? Deze twee varianten werken in de basis hetzelfde als hun `First` -broertjes, maar leggen de lat nóg hoger: ze eisen dat er **exact één item** voldoet aan de voorwaarde. Bestaan er in de lijst per ongeluk twee laptops met dezelfde `Id` ? Dan gooit `Single(OrDefault)` direct een `Exception`. Dit is een ideale ingebouwde controle voor unieke waarden zoals ID's of rijksregisternummers!

ActionResult<T> & Status codes

Vaak is het geen goed idee om het resultaat van een endpoint rechtstreeks aan een gebruiker terug te geven. Wanneer je een methode schrijft die een `List<Laptop>` teruggeeft, ben je beperkt. Wat als de lijst leeg is? Wat als de server een interne fout heeft? De client krijgt dan vaak een kale 200 OK met een lege lijst, of erger: een technisch foutbericht dat de gebruiker niet begrijpt.

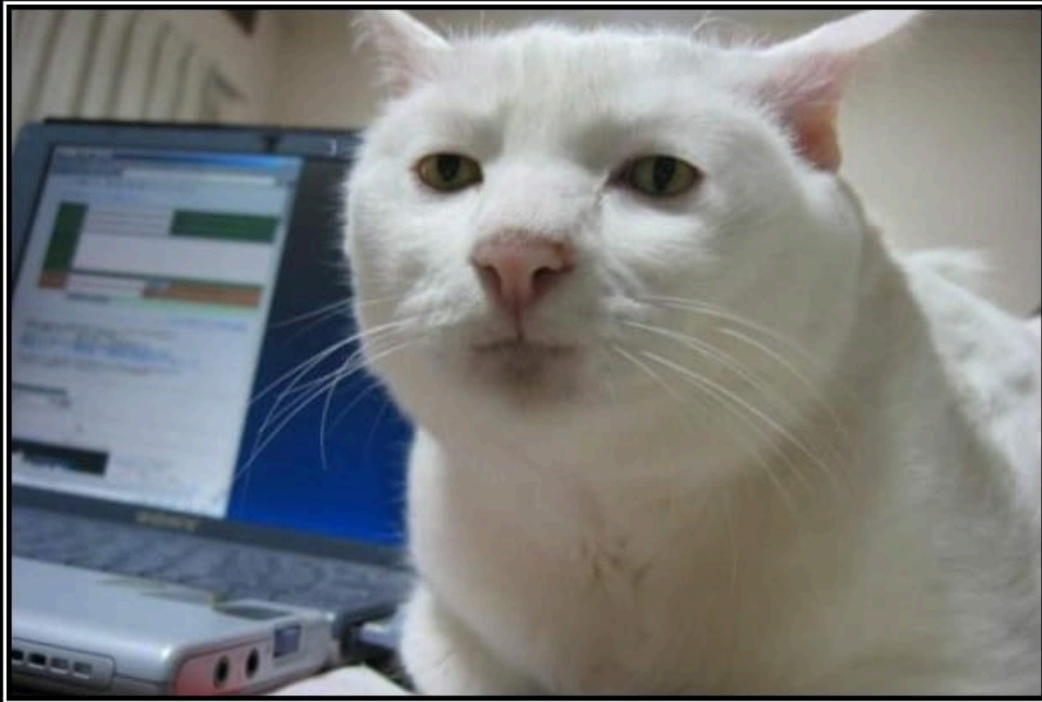
Door `ActionResult<T>` te gebruiken, geef je jezelf de mogelijkheid om HTTP Statuscodes te communiceren. Dit is de universele taal van het internet. Vaak is dit de enige manier dat een client te weten kan komen of zijn request succesvol was. **Probeer dus steeds zo'n duidelijk mogelijke status code te voorzien.**

Hieronder vind je een overzicht van de meest voorkomende HTTP status codes en wanneer je deze gebruikt in een applicatie. De codes in deze tabel zouden door elke zichzelf respecterende developer gekend moeten zijn.

Code	Naam	C# return type	Wanneer gebruik je dit?
200	OK	<code>Ok()</code>	Alles is gelukt en hier is je gevraagde data.
201	Created	<code>CreatedAtAction()</code>	Gebruikt bij <code>POST</code> : De nieuwe data is succesvol opgeslagen.
204	No Content	<code>NoContent()</code>	Succesvol, maar geen data om terug te sturen (vaak bij <code>DELETE</code> en <code>PUT</code>).
400	Bad Request	<code>BadRequest()</code>	Foutieve input van de client.
404	Not Found	<code>NotFound()</code>	Waarschijnlijk de meest bekende foutcode ter wereld. De gevraagde resource bestaat niet of kan niet gevonden worden.
500	Internal Server Error	<code>Exception</code>	Er zit een bug in jouw C#-code.

Tips

Neem eens een kijkje op http.cat voor een humoristisch overzicht van alle HTTP status codes. 🐱



200
OK

Over het algemeen:

- **200 range:** Request succesvol.
- **400 range:** Klant maakte een fout.
- **500 range:** Wij maakten een fout.

Laten we onze ProductController.cs herschrijven op een professionele manier met `ActionResult<T>` . Op deze manier kunnen we makkelijk duidelijk maken wanneer een request succesvol is afgerond en wanneer er een fout opgetreden is.

```
1  using Microsoft.AspNetCore.Mvc;
2  using WebAPIDemo.Models;
3
4  namespace WebAPIDemo.Controllers;
5
6  [ApiController]
7  [Route("api/[controller]")] // Route wordt: api/product
8  public class ProductController : ControllerBase
9  {
```

csharp

```
10     // Onze tijdelijke "database" in het geheugen
11     private List<Laptop> laptops = new List<Laptop>
12     {
13         new Laptop { Id = 1, Merk = "Dell", Processor = "Intel i7",
14         RamInGB = 16, Prijs = 1200.00, GPU = "Iris Xe" },
15         new Laptop { Id = 2, Merk = "Apple", Processor = "M3 Pro",
16         RamInGB = 18, Prijs = 2500.00, GPU = "Apple GPU" },
17         new Laptop { Id = 3, Merk = "Lenovo", Processor = "AMD Ryzen 7",
18         RamInGB = 32, Prijs = 1400.00, GPU = "RTX 4060" }
19     };
20
21     // 1. GET: Alle laptops ophalen
22     // Route: GET api/laptops
23     [HttpGet]
24     public ActionResult<List<Laptop>> GetLaptops()
25     {
26         // We sturen een 200 OK status terug samen met de lijst
27         return Ok(laptops);
28     }
29
30     // 2. GET: Eén specifieke laptop op basis van Id
31     // Route: GET api/laptops/1
32     [HttpGet("{id}")]
33     public ActionResult<Laptop> GetLaptopById(int id)
34     {
35         // Zoek de laptop in de lijst die de gevraagde Id heeft
36         var laptop = laptops.FirstOrDefault(x => x.Id == id);
37
38         // Als de laptop niet bestaat, stuur dan een 404 terug
39         if (laptop == null)
40         {
41             return NotFound($"Helaas, we konden geen laptop vinden met Id
42 {id}.");
43         }
44
45         return Ok(laptop);
46     }
47 }
```

HttpPost - Data toevoegen

Tot nu toe hebben we enkel data van de server opgevraagd. In de praktijk wil een client natuurlijk ook gegevens *naar* de server sturen. Denk aan het aanmaken van een account, het plaatsen van een bericht op social media, of het toevoegen van een nieuw product aan een webshop. Om data naar de server te sturen, maken we gebruik van de `HttpPost` verb.

In tegenstelling tot een `GET`-verzoek, waarbij parameters *meestal in de URL zichtbaar zijn*, verpakt een `POST`-verzoek de gegevens veilig in de **Body** (het 'lijf') van het HTTP-verzoek. Deze data wordt door de client verzonden als JSON, en ASP.NET Core zet dit achter de schermen automatisch om naar ons C# Laptop-model. Dit proces noemen we `Model Binding`.

Voeg de volgende methode toe aan je `ProductController.cs`:

```
1 // 3. POST: Een nieuwe laptop toevoegen csharp
2 // Route: POST api/laptops
3 [HttpPost]
4 public ActionResult<Laptop> CreateLaptop(Laptop nieuweLaptop)
5 {
6     // Omdat we geen database hebben die automatisch ID's genereert,
7     // zoeken we zelf de hoogste ID en tellen we er 1 bij op.
8     int nieuweId = laptops.Max(x => x.Id) + 1;
9     nieuweLaptop.Id = nieuweId;
10
11     // Voeg de nieuwe laptop toe aan onze in-memory lijst
12     laptops.Add(nieuweLaptop);
13
14     // We retourneren een 201 Created statuscode.
15     // Dit vertelt de client dat het object succesvol is aangemaakt én
16     // waar het te vinden is.
17     return CreatedAtAction(nameof(GetLaptopById), new { id =
        nieuweLaptop.Id }, nieuweLaptop);
}
```

CreatedAtAction

Je zult merken dat we hier niet `Ok()` gebruiken, maar `CreatedAtAction()`. Volgens de officiële REST-richtlijnen hoort een succesvolle POST een `201 Created` statuscode te retourneren.

Deze methode doet drie dingen voor je:

- Het verandert de HTTP-statuscode naar `201 Created`.
- Het stuurt de zojuist aangemaakte Laptop (inclusief zijn nieuwe `Id`) terug in de body, zodat de frontend dit direct kan tonen.
- Het voegt een `Location`-header toe aan het HTTP-antwoord. Dit is een URL die de client precies vertelt waar hij de gloednieuwe laptop kan opvragen (bijv. `api/laptops/4`).

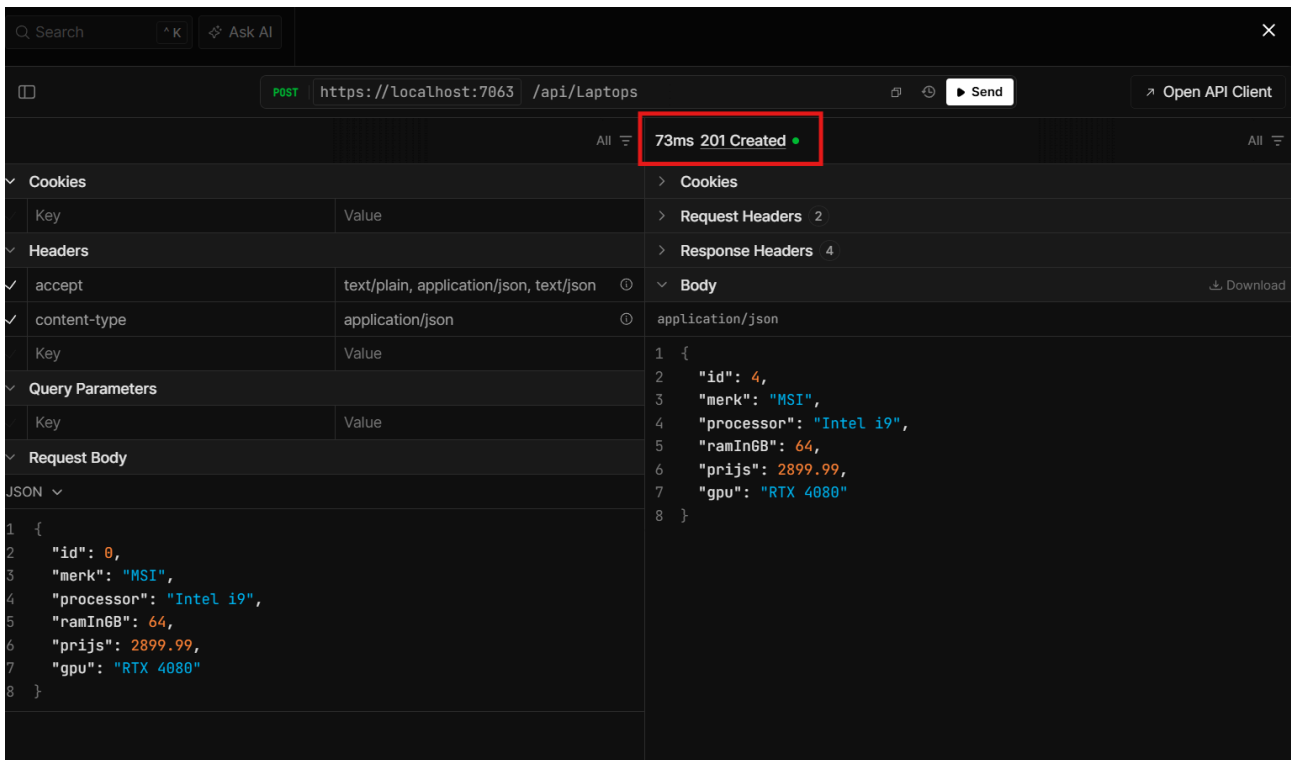
Testen in Scalar

Omdat je een POST-verzoek niet via de gewone adresbalk van je browser kunt simuleren, komt Scalar hier als geroepen:

- Open Scalar en klik op `POST /api/laptops`.
- Je zult zien dat Scalar automatisch een voorbeeld-JSON klaarzet in de Request Body.
- Vul de velden in met een nieuwe laptop (je mag de `id` op `0` laten staan, onze code herkent deze immers zelf):

```
1  {
2      "id": 0,
3      "merk": "MSI",
4      "processor": "Intel i9",
5      "ramInGB": 64,
6      "prijs": 2899.99,
7      "gpu": "RTX 4080"
8  }
```

json



- Klik op Send. Als alles goed gaat, zie je een statuscode `201 Created` verschijnen.
- Voer nu de `GET /api/laptops` methode opnieuw uit. Je zult zien dat de MSI-laptop nu achteraan in de lijst staat.

The screenshot shows an API client interface with the following details:

- URL:** `https://localhost:7063 /api/Laptops`
- Method:** GET
- Status:** 200 OK
- Response Headers:**
 - `Content-Type: application/json`
- Response Body:**

```

1 [
2   {
3     "id": 1,
4     "merk": "Dell",
5     "processor": "Intel i7",
6     "ramInGB": 16,
7     "prijs": 1200,
8     "gpu": "Iris Xe"
9   },
10  {
11    "id": 2,
12    "merk": "Apple",
13    "processor": "M3 Pro",
14    "ramInGB": 18,
15    "prijs": 2500,
16    "gpu": "Apple GPU"
17  },
18  {
19    "id": 3,
20    "merk": "Lenovo",
21    "processor": "AMD Ryzen 7",
22    "ramInGB": 32,
23    "prijs": 1400,
24    "gpu": "RTX 4060"
25  },
26  {
27    "id": 4,
28    "merk": "MSI",
29    "processor": "Intel i9",
30    "ramInGB": 64,
31    "prijs": 2899.99,
32    "gpu": "RTX 4080"
33  }
34 ]

```

Notitie

Omdat de data in het RAM-geheugen staat, ben je alles kwijt zodra je de applicatie herstart. Voor nu is dit echter de perfecte, veilige zandbak om te experimenteren. Later vervangen we deze data door echte data uit een database.

HttpPut - Data updaten

Nu we data kunnen lezen en schrijven, missen we nog een manier om fouten te corrigeren. Stel dat de prijs van een laptop verandert, of we hebben een typfout gemaakt in de naam van de GPU. Hiervoor gebruiken we de `HttpPut` verb.

Bij een `PUT`-verzoek sturen we twee dingen mee:

- De `id` van de laptop die we willen aanpassen (in de URL).
- De nieuwe, volledige gegevens van de laptop (in de Request Body).

Voeg de volgende methode toe aan je controller:

```
1 // 4. PUT: Een bestaande laptop volledig bijwerken csharp
2 // Route: PUT api/laptops/1
3 [HttpPut("{id}")]
4 public ActionResult UpdateLaptop(int id, Laptop bijgewerkteLaptop)
5 {
6     // 1. Zoek de laptop die momenteel in de lijst staat
7     var bestaandeLaptop = laptops.FirstOrDefault(x => x.Id == id);
8
9     // 2. Als de laptop niet bestaat, sturen we een 404 Not Found
10    if (bestaandeLaptop == null)
11    {
12        return NotFound($"Kan geen laptop bijwerken met Id {id}, omdat
13        deze niet bestaat.");
14    }
15
16    // 3. Overschrijf de eigenschappen met de nieuwe data uit de body
17    bestaandeLaptop.Merk = bijgewerkteLaptop.Merk;
18    bestaandeLaptop.Processor = bijgewerkteLaptop.Processor;
19    bestaandeLaptop.RamInGB = bijgewerkteLaptop.RamInGB;
20    bestaandeLaptop.Prijs = bijgewerkteLaptop.Prijs;
21    bestaandeLaptop.GPU = bijgewerkteLaptop.GPU;
22
23    // 4. Update succesvol! We sturen een 204 No Content terug.
24    return NoContent();
25 }
```

Info

Merk op dat deze methode zowel data via de url meegeeft ("{id}") om een bestaand object op te halen en data in de body om het object te wijzigen.

Notitie

Let steeds op met de data die je via de URL meegeeft. Deze is immers door iedereen leesbaar.

Waarom 204 No Content?

In tegenstelling tot `GET` en `POST` geven we bij een succesvolle `PUT` vaak geen data terug. De client heeft de gewijzigde gegevens immers zojuist zelf naar ons opgestuurd! De statuscode 204 No Content is de perfecte manier om te zeggen: "Het is gelukt, ik heb je wijzigingen doorgevoerd, maar ik heb verder niets meer te tonen."

HttpDelete - Data verwijderen

De laatste stap van onze CRUD-dans is de meest rigoureuze: gegevens definitief weggooien. Hiervoor gebruiken we de `HttpDelete` verb.

Gelukkig is deze methode qua code vrij eenvoudig. We hebben enkel de `Id` in de URL nodig om te weten welk object er uit de lijst verwijderd moet worden.

```
1 // 5. DELETE: Een laptop verwijderen csharp
2 // Route: DELETE api/laptops/1
3 [HttpDelete("{id}")]
4 public ActionResult DeleteLaptop(int id)
5 {
6     // 1. Zoek de laptop in onze lijst
7     var laptop = laptops.FirstOrDefault(x => x.Id == id);
8
9     // 2. Bestaat hij niet? Dan kunnen we hem ook niet verwijderen ->
10    404!
11    if (laptop == null)
12    {
13        return NotFound($"Kan laptop met Id {id} niet verwijderen, omdat
14        deze niet is gevonden.");
15    }
16
17    // 3. Verwijder het object uit de lijst
18    laptops.Remove(laptop);
19
20    // 4. Succesvol verwijderd -> We sturen een 204 No Content terug
    return NoContent();
}
```

Net als bij de `PUT` -methode gebruiken we hier een `204 No Content` . De laptop is immers succesvol vernietigd, dus er valt simpelweg niets meer te tonen.

Info

De illusie van verwijderen In de echte wereld is het permanent weggooien van data (een zogenaamde `hard delete`) ontzettend zeldzaam. Verwijderen is namelijk heel definitief. Als je geen waterdichte back-ups hebt, is die data onherroepelijk verloren.

Denk er eens logisch over na: stel dat een webshop een bepaald model laptop niet meer verkoopt. Gooi je die dan uit de database? Wat gebeurt er dan met de bestelhistorie van de honderden klanten die die laptop ooit **gekocht** hebben? Als hun factuur verwijst naar een product-ID dat plotseling niet meer bestaat, krijg je foutmeldingen en klopt de boekhouding niet meer.

In de praktijk gebruiken we daarom meestal een `soft delete` . Hierbij wordt een item niet écht vernietigd, maar krijgt het in de database het label 'non-actief'. In je C#-model los je dit later op door bijvoorbeeld een property `bool IsDeleted` of `bool IsActive` mee te geven. Bij een `DELETE`-request zet je die simpelweg op `true` of `false` . De data blijft behouden maar je API filtert het weg zodat de client het niet meer ziet.

Info

Notitie

Er is één grote uitzondering: de **GDPR-wetgeving** (*Privacywetgeving*). Burgers hebben het wettelijke 'recht om vergeten te worden'. Als een klant expliciet eist dat zijn of haar persoonsgegevens worden verwijderd, *moet* je deze data permanent wissen (of onomkeerbaar anonimiseren). Maar voor bedrijfsdata zoals producten, bestellingen en interne logs is een `soft delete` de absolute koning!

Je **ProductController** zou er als volgt moeten uitzien:

```
1  using Microsoft.AspNetCore.Mvc;
2  using WebAPIDemo.Models;
3
4  namespace WebAPIDemo.Controllers;
5
6  [ApiController]
7  [Route("api/[controller]")] // Route wordt: api/laptops
8  public class ProductController : ControllerBase
```

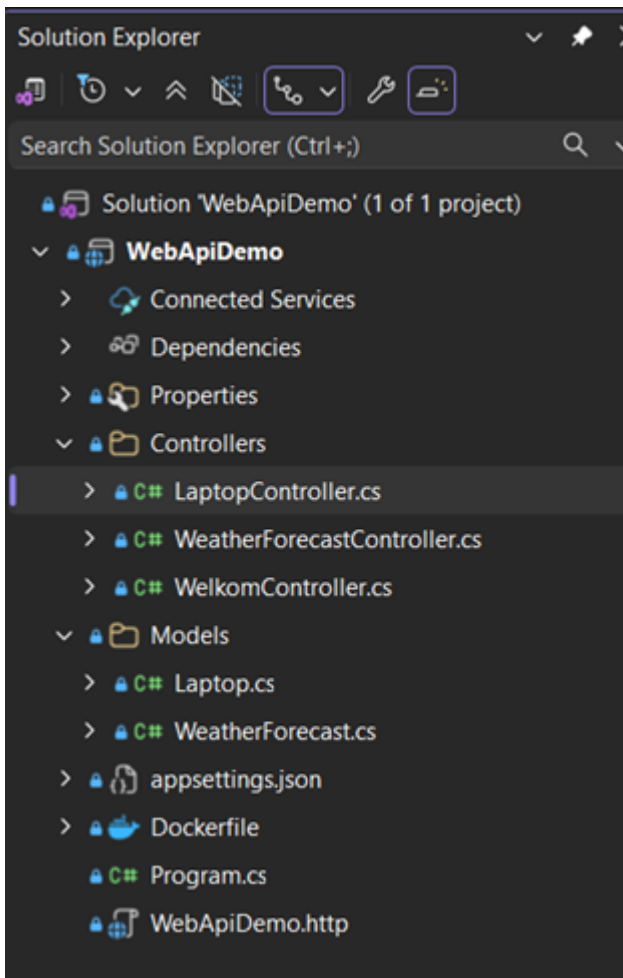
csharp

```
9      {
10         // Onze tijdelijke "database" in het geheugen
11         private static List<Laptop> laptops = new List<Laptop>
12         {
13             new Laptop { Id = 1, Merk = "Dell", Processor = "Intel i7",
14             RamInGB = 16, Prijs = 1200.00, GPU = "Iris Xe" },
15             new Laptop { Id = 2, Merk = "Apple", Processor = "M3 Pro",
16             RamInGB = 18, Prijs = 2500.00, GPU = "Apple GPU" },
17             new Laptop { Id = 3, Merk = "Lenovo", Processor = "AMD Ryzen 7",
18             RamInGB = 32, Prijs = 1400.00, GPU = "RTX 4060" }
19         };
20
21         // 1. GET: Alle laptops ophalen
22         // Route: GET api/laptops
23         [HttpGet]
24         public ActionResult<List<Laptop>> GetAlleLaptops()
25         {
26             // We sturen een 200 OK status terug samen met de lijst
27             return Ok(laptops);
28         }
29
30         // 2. GET: Eén specifieke laptop op basis van Id
31         // Route: GET api/laptops/1
32         [HttpGet("{id}")]
33         public ActionResult<Laptop> GetLaptopById(int id)
34         {
35             // Zoek de laptop in de lijst die de gevraagde Id heeft
36             var laptop = laptops.FirstOrDefault(x => x.Id == id);
37
38             // Als de laptop niet bestaat, stuur dan een 404 terug
39             if (laptop == null)
40             {
41                 return NotFound($"Helaas, we konden geen laptop vinden met Id
42 {id}.");
43             }
44
45             return Ok(laptop);
46         }
47
48         // 3. POST: Een nieuwe laptop toevoegen
49         // Route: POST api/laptops
50         [HttpPost]
51         public ActionResult<Laptop> CreateLaptop(Laptop nieuweLaptop)
```

```
52     {
53         // Omdat we geen database hebben die automatisch ID's genereert,
54         // zoeken we zelf de hoogste ID en tellen we er 1 bij op.
55         int nieuweId = laptops.Max(x => x.Id) + 1;
56         nieuweLaptop.Id = nieuweId;
57
58         // Voeg de nieuwe laptop toe aan onze in-memory lijst
59         laptops.Add(nieuweLaptop);
60
61         // We retourneren een 201 Created statuscode.
62         // Dit vertelt de client dat het object succesvol is aangemaakt
63         én waar het te vinden is.
64         return CreatedAtAction(nameof(GetLaptopById), new { id =
65         nieuweLaptop.Id }, nieuweLaptop);
66     }
67
68     // 4. PUT: Een bestaande laptop volledig bijwerken
69     // Route: PUT api/laptops/1
70     [HttpPut("{id}")]
71     public ActionResult UpdateLaptop(int id, Laptop bijgewerkteLaptop)
72     {
73         // 1. Zoek de laptop die momenteel in de lijst staat
74         var bestaandeLaptop = laptops.FirstOrDefault(x => x.Id == id);
75
76         // 2. Als de laptop niet bestaat, sturen we een 404 Not Found
77         if (bestaandeLaptop == null)
78         {
79             return NotFound($"Kan geen laptop bijwerken met Id {id},
80 omdat deze niet bestaat.");
81         }
82
83         // 3. Overschrijf de eigenschappen met de nieuwe data uit de body
84         bestaandeLaptop.Merk = bijgewerkteLaptop.Merk;
85         bestaandeLaptop.Processor = bijgewerkteLaptop.Processor;
86         bestaandeLaptop.RamInGB = bijgewerkteLaptop.RamInGB;
87         bestaandeLaptop.Prijs = bijgewerkteLaptop.Prijs;
88         bestaandeLaptop.GPU = bijgewerkteLaptop.GPU;
89
90         // 4. Update succesvol! We sturen een 204 No Content terug.
91         return NoContent();
92     }
93
94
```

```
95     // 5. DELETE: Een laptop verwijderen
96     // Route: DELETE api/laptops/1
97     [HttpDelete("{id}")]
98     public ActionResult DeleteLaptop(int id)
99     {
100         // 1. Zoek de laptop in onze lijst
101         var laptop = laptops.FirstOrDefault(x => x.Id == id);
102
103         // 2. Bestaat hij niet? Dan kunnen we hem ook niet verwijderen ->
104         404!
105         if (laptop == null)
106         {
107             return NotFound($"Kan laptop met Id {id} niet verwijderen,
108             omdat deze niet is gevonden.");
109         }
110
111         // 3. Verwijder het object uit de lijst
112         laptops.Remove(laptop);
113
114         // 4. Succesvol verwijderd -> We sturen een 204 No Content terug
115         return NoContent();
116     }
117 }
```

en je solution view zo:



Samenvatting

Je eerste REST API is een feit! Hoewel je data nu nog tijdelijk in het RAM-geheugen leeft, volgt jouw controller de wereldwijde HTTP-standaarden.

Kernconcepten in één zin

- **Controller** : De 'digitale ober' die HTTP-verzoeken opvangt, doorzet naar C#-methoden en het resultaat serveert.
- **Models** : Klassen (zoals Laptop.cs) die objecten uit de echte wereld structureren via **properties**.
- **JSON** : De universele teksttaal van het internet waarin data tussen client en server wordt uitgewisseld.
- **Model Binding** : De automatische vertaling van en naar JSON uit de **Request Body** naar een bruikbaar C#-object.

- **Route Parameters** : Variabelen in de URL (bijv. {id}) die .NET rechtstreeks aan je methode-parameters koppelt.
 - **ActionResult<T>** : Het retourtype waarmee je flexibel data en **HTTP-statuscodes** terugstuurt.
 - **Scalar** : Interactieve testomgeving om endpoints (GET , POST , PUT , DELETE) handmatig te testen.
-

Statuscodes

-  **200-range (Success) – "Alles ok!"**
 - 200 OK : Data succesvol opgehaald.
 - 201 Created : Item succesvol aangemaakt (standaard bij POST).
 - 204 No Content : Actie geslaagd, maar geen data terug te sturen (standaard bij PUT en DELETE).
-  **400-range (Client Error) – "Fout van de frontend!"**
 - 400 Bad Request : Ongeldige data verstuurd.
 - 404 Not Found : De opgevraagde URL of specifieke Id bestaat niet.
-  **500-range (Server Error) – "Paniek in de backend!"**
 - 500 Internal Server Error : Er zit een bug of onopgevangen crash in jouw C#-code.

Laatst bijgewerkt:09-09-2026, 08:52

Bijdragers: JorenSynaeveTM